

Contents

1	API 設計書 (メイン)	1
1.1	1. 文書概要	1
1.1.1	1.1 目的	1
1.1.2	1.2 対象範囲	1
1.1.3	1.3 版数	1
1.1.4	1.4 関連ドキュメント	2
1.2	2. API 一覧	2
1.2.1	2.1 機能と API の対応 (§ 6.3)	2
1.2.2	2.2 外部 I/F 一覧 (§ 8.1)	2
1.3	3. 共通仕様	3
1.3.1	3.1 API 共通仕様 (§ 8.2)	3
1.3.2	3.2 オーナー境界判定	3
1.3.3	3.3 幂等性キー	3
1.3.4	3.4 Webhook 共通仕様 (§ 8.7.3)	3
1.3.5	3.5 連携 IF エンドポイント命名規約 (§ 8.8)	3
1.3.6	3.6 認証ヘッダ	3
1.4	4. レート制限・キャパシティ (§ 11.7 リミット設計表の正本)	3
1.5	5. API 詳細	4
1.5.1	5.1 認証 API	4
1.5.2	5.2 利用者(メンバー)API	5
1.5.3	5.3 プロジェクト管理 API	7
1.5.4	5.4 FAQ 管理 API	9
1.5.5	5.5 ウィジェット API	10
1.5.6	5.6 未解決質問 API	11
1.5.7	5.7 チャット API	11
1.5.8	5.8 利用量・課金 API	11
1.5.9	5.9 お知らせ受信箱 API	12
1.5.10	5.10 規約・退会 API	12
1.5.11	5.11 AI 推論 IF(AnswerProvider 抽象化)	13
1.5.12	5.12 メール配信 IF(EmailProvider 抽象化)	13
1.5.13	5.13 連携 IF(受信側、メイン側エンドポイント)	13
1.5.14	5.14 外部 Webhook(Resend 等)	16
1.6	6. 連携 IF 通信仕様一覧	17
1.7	7. 未決事項	17
1.8	8. 変更履歴	17

1 API 設計書 (メイン)

1.1 1. 文書概要

1.1.1 1.1 目的

メインシステムのすべての API(管理 API / ウィジェット API / 連携 IF #1~#12 送信側 / 外部 Webhook) について、エンドポイント / 認証・認可 / リクエスト / レスポンス / ステータスコード / エラー仕様を一元化する。

1.1.2 1.2 対象範囲

- 対象: 利用者向け管理 API(認証 / プロジェクト / FAQ / 案件 / チャット / ウィジェット設定 / 利用量・課金 / ユーザー管理 / お知らせ / 規約)、エンドユーザー向けウィジェット API、AI 推論抽象化 IF、メール配信抽象化 IF、顧客管理システムとの連携 IF #1~#12(送信側)、外部 Webhook(Resend 等)
- 対象外: 運営者管理 API([02_運営者システム/個別設計書群/03_API 設計書.md](#))、Stripe Webhook 一次受信(運営者側 § 13)

1.1.3 1.3 版数

項目	値
版数	1.0
更新日	2026-05-17

1.1.4 1.4 関連ドキュメント

ドキュメント名	役割	参照先
索引	11 ドキュメント体系の俯瞰	00_ 索引.md
画面設計書	API を呼び出す画面	02_ 画面設計書.md
テーブル定義書	API が操作するテーブル	04_ テーブル定義書.md
認証・認可設計書	認証要否 / 認可判定ルール	09_ 認証認可設計書.md
エラー設計書	ステータスコード / エラー詳細	06_ エラー設計書.md
メッセージ一覧	エラー文言	07_ メッセージ一覧.md
課金・請求設計書	課金 API / IF #1 / IF #10	11_ 課金請求設計書.md
運営者側 API 設計書	連携 IF #1～#12 受信側	../02_ 運営者システム/個別設計書群/03_API 設計書.md

1.2 2. API 一覧

1.2.1 2.1 機能と API の対応 (§6.3)

機能	API エンドポイント	認可	関連画面
認証	/auth/register, /auth/login, /auth/logout, /auth/re-auth, /auth/password/reset-requests	公開 (login)、認証済み (他)	SCR-001, SCR-002, SCR-003
メール確認	/auth/email-verifications/{token}	トークン認証	SCR-023
管理者ユーザー管理	/members, /members/invite, /members/{id}/project-grants, /members/{id}/resend-invitation, /members/{id}, /projects/{id}/members, /projects/{id}/members/{accountId}	オーナー / プロジェクト管理者 (該当 PJ 範囲)	SCR-017, SCR-017-M1, SCR-010-M1
プロジェクト管理	/projects, /projects/{id}	オーナー専有	SCR-010, SCR-010-M1
FAQ 管理	/projects/{id}/faqs, /projects/{id}/faqs/{faqId}, /projects/{id}/faqs/import, /projects/{id}/faqs/export	オーナー / 該当 PJ の member+	SCR-012
ウィジェット	/widget/bootstrap, /widget/ask, /widget/feedback	end_user (公開キー + セッション)	ウィジェット
未解決質問	/inquiries, /inquiries/{id}, /inquiries/{id}/close	オーナー / 該当 PJ の member+	SCR-011
チャット	/inquiries/{code}/email-registration, /chat/rooms/{id}, /chat/rooms/{id}/messages	admin / end_user	SCR-013
通知	/webhooks/resend	署名検証のみ	(Resend Webhook)
利用量・課金	/billing/summary, /billing/subscription, /billing/invoices, /billing/budget	admin / オーナー専有	SCR-015
データ管理	/data/export	admin / オーナー専有	SCR-024
お知らせ受信箱	/me/announcements, /me/announcements/{id}, /me/announcements/{id}/read	admin 限定	SCR-021, SCR-022
規約	/terms/current, /terms/agree	admin	SCR-018, SCR-025
再入室	/widget/sessions/:token	トークン認証	SCR-027

1.2.2 2.2 外部 I/F 一覧 (§8.1)

カテゴリ	接続先	プロトコル	認証方式
顧客管理システム	運営者側システム	HTTPS + mTLS + JWT	連携 IF #1～#12
Stripe	課金決済	HTTPS	API キー、Webhook 署名
Resend	メール配信	HTTPS	API キー、Webhook 署名
Cloudflare Workers AI	AI 推論	HTTPS (内部)	自動認証
ウィジェット埋込元サイト	顧客サイト	HTTPS + CORS	公開キー + セッショントークン

1.3 3. 共通仕様

1.3.1 3.1 API 共通仕様 (§8.2)

項目	値
Base URL	/api/v1(管理)/ /widget/v1(ウィジェット)/ /internal/admin-integration/v1(内部連携)
データ形式	JSON
エラー形式	RFC 7807(application/problem+json)
日時	ISO 8601 + 末尾 Z(UTC)
ID 形式	ULID(26 文字)、Stripe ID 例外
ページング	カーソル方式 (cursor, limit 50~200)
CSRF	Double Submit Cookie 検証 (状態変更 API)
Cookie	Secure; HttpOnly; SameSite=Lax; Path=/

1.3.2 3.2 オーナー境界判定

すべての操作で `actor.owner_account_id == target.owner_account_id` を必須 (認証・認可設計書 09 § 5.2)。違反時は **404 偽装** (エラー設計書 06 § 5.3)。

1.3.3 3.3 冪等性キー

書き込み系 API は **Idempotency-Key** ヘッダ (ULID 推奨、24h 保管) を受け付ける。重複時は既存処理結果を 200 で返却 (リトライ安全性)。

1.3.4 3.4 Webhook 共通仕様 (§8.7.3)

項目	仕様
署名検証	HMAC-SHA256(送信元との共有 Secret)
冪等性	event_id で重複処理防止
リトライ	3 回 (指数 BO: 1m → 4m → 16m)、超過で DLQ
DLQ 保持	4 日 (Cloudflare Queues)、その後 R2 退避 30 日
タイムアウト	30 秒 (超過で 5xx 返却)

1.3.5 3.5 連携 IF エンドポイント命名規約 (§8.8)

- ・ 受信: `/internal/admin-integration/v1/{resource}/{action}`
- ・ 送信: `https://<admin-host>/internal/main-integration/v1/...`
- ・ リソース: snake_case 単数形
- ・ 動詞: POST(状態変更)

1.3.6 3.6 認証ヘッダ

ヘッダ	用途	例
Cookie: session=<token>	利用者セッション	管理 API
Authorization: Bearer <wst_*>	ウィジェットセッション	ウィジェット API
Authorization: Bearer <JWT>	連携 IF	内部連携
X-CSRF-Token: <token>	CSRF 対策	状態変更系
X-Inquiry-Token: <token>	エンドユーザー再入室	/widget/sessions/token
Idempotency-Key: <ULID>	冪等性	書き込み系

1.4 4. レート制限・キャパシティ (§11.7 リミット設計表の正本)

項目	警告レベル	拒否レベル	契約上書き	超過時の挙動	正 本
同時アクティブ契約	150 でシャーディング着手	200	不可	新規契約受付一時停止 + 運営者通知	NFR-110
FAQ 件数 (契約共通)	8,000(80%)	12,000(120%)	不可	拒否レベルで 429	FR-046
FAQ 質問文字数	-	500 文字	不可	超過は 400	FR-046
FAQ 回答文字数	-	5,000 文字	不可	超過は 400	FR-046
月間質問数	80%(通知)	100%(事後課金、拒否しない)	契約別上書き可	125% で追加制限	FR-122
Workers AI 月間コスト	80%	100%(停止)	契約別上書き可	同上	FR-122
チャット投稿 (EU)	-	10 件/分、2,000 字/msg	不可	429、UI 警告	FR-090
チャット投稿 (admin)	-	無制限 (運営者調整可)	可 (IF #5)	429	FR-090
公開 API /wid-get/ask	-	60 req/min/IP	可 (IF #5)	429	FR-128
公開 API /wid-get/bootstrap	-	30 req/min/IP	可 (IF #5)	429	FR-128
ログイン失敗	-	5 回/15 分 (IP × user_id ロック)	不可	アカウントロック	FR-007
管理者ユーザー数	80%	100%	可 (プラン依存)	429、通知	FR-021
プロジェクト数	平均 × 3 超過時通知	-	-	急増検知通知のみ	FR-035
同時オンライン管理者	-	1,000 名	-	503 + Retry-After	NFR-114
ウィジェット同時接続 (EU)	-	10,000 並列	-	503 + Retry-After	NFR-114
D1 容量	8GB(80%)	10GB	-	高優先度アラート + シャーディング検討	NFR-115
Workers CPU 時間	50ms 目標 / 100ms 警告	-	-	100ms 超過が 1h で 1% 超で通知	NFR-116

詳細は [11_課金請求設計書.md § 8](#) 80/100/125% 三段階アクション参照。

1.5 5. API 詳細

1.5.1 5.1 認証 API

1.5.1.1 5.1.1 POST /auth/signup(新規登録)

項目	内容
機能 ID	F-001
認証要否	不要
関連画面	SCR-002
関連テーブル	accounts, terms_agreements

リクエスト:

```
{
  "email": "admin@example.com",
  "password": "P@ssw0rd123!",
  "passwordConfirm": "P@ssw0rd123!",
  "agreedTerms": true,
  "agreedPrivacy": true,
```

```
    "turnstileToken": "..."  
  }
```

レスポンス (202):

```
{ "ok": true }
```

エラー: 400 **VALIDATION_ERROR**(E-INPUT-*), 400 **TURNSTILE_FAILED**

1.5.1.2 5.1.2 POST /auth/login

項目	内容
機能 ID	F-002
認証要否	不要
関連画面	SCR-001
関連テーブル	accounts, sessions

リクエスト:

```
{  
  "email": "admin@example.com",  
  "password": "...",  
  "turnstileToken": "..."  
}
```

レスポンス (200):

```
{  
  "accountId": "01HZ...",  
  "role": "admin",  
  "requireTermsAgreement": false,  
  "activeSessions": [  
    { "id": "...", "ipAddress": "203.0.113.x", "createdAt": "..." }  
  ]  
}
```

エラー: 401 **INVALID_CREDENTIALS**(E-AUTH-CREDENTIAL), 423 **LOCKED_OUT**(E-AUTH-LOCKED), 400 **TURNSTILE_REQUIRED**, 403 **CONTRACT_SUSPENDED**(E-BILL-CONTRACT-SUSPENDED)

1.5.1.3 5.1.3 POST /auth/logout | 認証 | Cookie + CSRF | | レスポンス | 200 { "ok": true } |

1.5.1.4 5.1.4 POST /auth/password-reset-request(パスワード再設定要求) | 機能 ID | F-004 | | 認証要否 | 不要 | | 関連画面 | SCR-003 |

リクエスト: { "email": "admin@example.com", "turnstileToken": "..." } レスポンス (202): { "ok": true } (列挙攻撃対策で存在有無を返さない)

1.5.1.5 5.1.5 POST /auth/re-auth(再認証) | 機能 ID | F-005 | | 認証要否 | 要 |

リクエスト: { "password": "..." } レスポンス (200): { "ok": true, "token": "reauth_..." } エラー: 401 **INVALID_PASSWORD**

1.5.1.6 5.1.6 POST /auth/email-verifications/{token}(メール確認) | 機能 ID | F-003 | | 関連画面 | SCR-023 |

トークン形式 = email_verify purpose の access_tokens.token_hash 検証。レスポンス (200): { "accountId": "...", "verifiedAt": "..." } エラー: 410 **TOKEN_EXPIRED**(E-AUTH-TOKEN-EXPIRED)

1.5.2 5.2 利用者 (メンバー)API

1.5.2.1 5.2.1 GET /members | 機能 ID | F-015 | | 必要権限 | オーナー / プロジェクト管理者 (該当 PJ 範囲のメンバーのみ可視) | | 関連画面 | SCR-017 |

レスポンス (200):

```
{
  "items": [
    {
      "id": "01HZ...",
      "email": "member@...",
      "status": "active|pending_activation",
      "role": "admin",
      "is_owner": false,
      "projectGrants": [
        { "projectId": "proj_1", "role": "admin" },
        { "projectId": "proj_2", "role": "member" }
      ],
      "lastLoginAt": "2026-05-13T10:00:00Z",
      "invitationExpiresAt": "2026-05-20T..."
    }
  ],
  "nextCursor": "01HZ..." | null
}
```

非オーナーが呼び出した場合、`items[]` は「操作者自身が `admin` を持つプロジェクトに割当のあるメンバー」に絞られ、各 `projectGrants[]` も操作者の `admin` 範囲のプロジェクトのみが見える。

1.5.2.2 5.2.2 POST /members/invite | 認証 | Cookie + CSRF + 再認証必須 | | 必要権限 | オーナー / プロジェクト管理者 (`projectGrants[*].projectId` の各プロジェクトで操作者自身が `admin` 保持) | | 関連画面 | SCR-017-M1 |

リクエスト:

```
{
  "email": "newmember@...",
  "displayName": " 新規メンバー",
  "projectGrants": [
    { "projectId": "proj_1", "role": "admin" },
    { "projectId": "proj_2", "role": "member" }
  ]
}
```

- `projectGrants` は 0 件以上 (0 件 = ダッシュボードのみ利用可能なメンバー、FR-015d)
- 操作者が非オーナーの場合、`projectGrants[*].projectId` の各プロジェクトで操作者自身が `admin` ロール保持を検証 (違反は 403 `PROJECT_ACCESS_DENIED`)
- 同一 `projectId` の重複は 400 `VALIDATION_ERROR`

レスポンス (201):

```
{
  "id": "01HZ...",
  "status": "pending_activation",
  "invitationToken": "invite_...",
  "expiresAt": "2026-05-20T..."
}
```

エラー: 409 `ALREADY_EXISTS`(FR-021c)、400 `VALIDATION_ERROR`、403 `PROJECT_ACCESS_DENIED`、403 `PERMISSION_DENIED`(オーナー専有)

1.5.2.3 5.2.3 POST /members/{id}/project-grants(全体上書き) | 認証 | Cookie + CSRF + 再認証必須 | | 必要権限 | オーナー専有 (契約全体のロール一括変更) |

メンバーのプロジェクト別ロール割当を全体上書きする。プロジェクト管理者は本 API を呼び出せない代わりに `POST /projects/{id}/members/DELETE` `/projects/{id}/members/{accountId}` を用いて該当プロジェクト範囲のみ操作する。

リクエスト:

```
{
  "projectGrants": [
    { "projectId": "proj_1", "role": "admin" },
  ]
}
```

```

    { "projectId": "proj_2", "role": "member" }
  ]
}

```

- ・送られた **projectGrants** 全体で **上書き** (差分計算は API 側で実施し監査ログ記録)
- ・空配列 = 全プロジェクト割当を解除 (ダッシュボードのみ利用可)
- ・本変更で「あるプロジェクトの最後の **admin** を外す」結果になる場合は 403 **LAST_ADMIN_PROTECTED** (E-AUTHZ-LAST-ADMIN-PROTECTED) で拒否

レスポンス (200): { "id": "...", "projectGrants": [...], "updatedAt": "..." } エラー: 404 **NOT_FOUND** (オーナー境界違反、E-AUTHZ-OWNER-BOUNDARY)、403 **OWNER_PROTECTED**、403 **LAST_ADMIN_PROTECTED**、403 **SELF_MUTATION_FORBIDDEN** (自分が **admin** を持つ最後の PJ で自身を **member** 化 / 割当解除)

1.5.2.4 5.2.4 POST /projects/{id}/members (プロジェクト単発招待) | 認証 | Cookie + CSRF + 再認証必須 | | 必要権限 | オーナー / 該当プロジェクトの **admin** ロール保持メンバー | | 関連画面 | SCR-017-M1 (プロジェクト管理者の招待操作) |

該当プロジェクト 1 件に対する単発招待。既存メンバーアカウントの場合は **account_project_grants** 行のみ追加 (指定ロール)。新規メールアドレスの場合は **accounts** を **pending_activation** で作成 + **account_project_grants** を同時 INSERT + 招待メール送信 (7 日有効、**access_tokens.purpose='activation'**)。

リクエスト:

```

{
  "email": "newmember@...",
  "displayName": " 新規メンバー",
  "role": "admin"
}

```

レスポンス (201): { "accountId": "01HZ...", "status": "active|pending_activation", "expiresAt": "..." | null } エラー: 409 **ALREADY_EXISTS_IN_PROJECT** (既に該当プロジェクトに割当あり)、400 **VALIDATION_ERROR**、403 **PROJECT_ACCESS_DENIED**

1.5.2.5 5.2.5 DELETE /projects/{id}/members/{accountId} (プロジェクトからの離脱) | 認証 | Cookie + CSRF + 再認証必須 | | 必要権限 | オーナー / 該当プロジェクトの **admin** ロール保持メンバー |

当該プロジェクトの **account_project_grants** 行のみ削除する (アカウント本体は削除しない)。削除後に他プロジェクト割当が 0 件かつ **is_owner=0** となっても **accounts** は維持 (ダッシュボードのみ利用可)。

エラー: 403 **LAST_ADMIN_PROTECTED** (本人が当該プロジェクトの最後の **admin** で、結果として **admin** が 0 件になる場合)、403 **SELF_MUTATION_FORBIDDEN** (自分が **admin** を持つ最後のプロジェクトでの自己離脱)、404 **NOT_FOUND**

1.5.2.6 5.2.6 DELETE /members/{id} / POST /members/{id}/resend-invitation | 認証 | Cookie + CSRF + 再認証必須 | | 必要権限 | **DELETE /members/{id}** は **オーナー専有** (アカウント完全削除)。**POST /members/{id}/resend-invitation** は **オーナー / 該当プロジェクトの admin 保持メンバー** | | 関連画面 | SCR-017-M1 (削除・招待再送) |

DELETE /members/{id} は招待中 (**pending_activation**)・有効 (**active**) いずれの状態でも実行可能。実行時は対象メンバーの全セッションを失効。プロジェクト管理者は本 API を呼び出せず、該当プロジェクトからの「離脱」 (**DELETE /projects/{id}/members/{accountId}**) で対応する。

POST /members/{id}/resend-invitation は **status='pending_activation'** のメンバーのみ対象。旧 **access_tokens.purpose='activation'** を失効させ新規トークン (7 日有効) を発行。

エラー: 403 **OWNER_PROTECTED** (オーナー対象)、403 **SELF_MUTATION_FORBIDDEN** (自分自身対象)、403 **PERMISSION_DENIED** (オーナー専有を非オーナーが呼出)、404 **NOT_FOUND**

1.5.3 5.3 プロジェクト管理 API

1.5.3.1 5.3.1 GET /projects | 機能 ID | F-034 | | 関連画面 | SCR-010 |

レスポンス (200):

```
{
  "items": [
    {
      "id": "01HZ...",
      "name": "...",
      "status": "active|deleted",
      "createdAt": "...",
      "faqCount": 42,
      "widgetKey": "pk_live...",
      "allowedDomains": ["example.com", "*.example.com"],
      "contactEmail": "contact@...",
      "contactEmailVerifiedAt": "..." | null,
      "ipAllowlist": ["203.0.113.0/24", "2001:db8::/32"]
    }
  ],
  "nextCursor": null
}
```

1.5.3.2 5.3.2 POST /projects(新規作成) | 認証 | Cookie + CSRF | | 必要権限 | **オーナー専有** (プロジェクト設定全般を含めオーナーのみ実行可) | | 関連画面 | SCR-010-M1(新規作成モード)|

リクエスト:

```
{
  "name": " 新規プロジェクト",
  "description": " 説明文",
  "allowedDomains": ["example.com"],
  "ipAllowlist": [],
  "initialAdmins": {
    "selfAsAdmin": true,
    "invitees": [
      { "email": "admin1@example.com", "displayName": " 管理者 A", "role": "admin" },
      { "email": "member1@example.com", "displayName": " メンバー A", "role": "member" }
    ]
  }
}
```

initialAdmins 仕様 (FR-030a / FR-015e):

- **selfAsAdmin=true** のとき、オーナー自身を当該プロジェクトの管理者として扱う (**account_project_grants** は OWNER 行を作らないため、認可は **isOwner** bypass で実現される。論理上「オーナーが管理者」)
- **invitees[]** は 0 件以上。各要素は **email / displayName / role(admin / member)** を持つ
- バリデーション: **selfAsAdmin=false** のときは **invitees[]** に **role='admin'** が 1 件以上必須。違反は 400 **VALIDATION_ERROR**(field: "initialAdmins"、メッセージは「最低 1 名の管理者が必要です」)
- **invitees[]** の **email** は同一オーナー配下の既存有効・招待中アカウントと重複不可 (FR-021c、409 **ALREADY_EXISTS**)、リスト内重複も不可 (400)
- 新規 **accounts** 行は **pending_activation** で作成、**account_project_grants** を同時 INSERT、**access_tokens.purpose='activation'** (7 日有効) を発行、招待メール送信を Queue 投入。既存メンバーアカウントが招待された場合は **account_project_grants** 追加のみ + 通知メール

レスポンス (201):

```
{
  "id": "01HZ...",
  "widgetKey": "pk_live_abc123...",
  "keyExpiresAt": "2027-05-13T...",
  "invitations": [
    { "accountId": "01HZ...", "email": "admin1@example.com", "status": "pending_activation", "e"
  ]
}
```

エラー: 400 **VALIDATION_ERROR**(name / initialAdmins 必須違反)、409 **DUPLICATE_NAME / ALREADY_EXISTS**、403 **E-AUTHZ-OWNER-ONLY**

1.5.3.3 5.3.3 PATCH /projects/{id} / DELETE /projects/{id} | 認証 | Cookie + CSRF(削除は再認証必須) | 必要権限 | **オーナー専有** | エラー | 404 NOT_FOUND(オーナー境界違反偽装)、403 E-AUTHZ-OWNER-ONLY |

リクエスト (PATCH、部分更新):

```
{
  "name": "...",
  "allowedDomains": [...],
  "contactEmail": "...",
  "ipAllowlist": ["203.0.113.0/24"]
}
```

DELETE /projects/{id} 実行時の挙動 (FR-030b):

1. 当該プロジェクトの **account_project_grants** を全行削除
2. 削除後、他プロジェクト割当が 0 件かつ **is_owner=0** となったメンバーの **accounts** を物理削除 (孤立メンバー cleanup)+ 全セッション失効
3. プロジェクト本体および紐づくデータ (FAQ / 質問ログ / 案件 / チャット 等) を削除 / 匿名化 (削除モードに従う)

上記 1-3 はアプリ層のトランザクション内で実装する。

1.5.3.4 5.3.3a PATCH /projects/{id}/ip-allowlist(プロジェクト単位 IP 許可リスト更新) | 機能 ID | F-179 / F-330 | | 関連画面 | SCR-010-M1 | | 必要権限 | **オーナー専有** (プロジェクト設定の一部) | | 認証 | Cookie + CSRF(再認証不要。設定対象は管理画面ではなくウィジェットのため) |

リクエスト:

```
{
  "ipAllowlist": ["203.0.113.0/24", "2001:db8::/32"]
}
```

レスポンス (200): { "ipAllowlist": ["203.0.113.0/24", "2001:db8::/32"], "updatedAt": "..."

評価対象スコープ:

- ・適用される: 当該プロジェクトの FAQ ウィジェット系エンドポイント (POST /widget/v1/bootstrap、POST /widget/v1/questions、POST /widget/v1/chat/*、POST /widget/v1/inquiry/reenter 等、プロジェクト ID または **pk_live_*** キーがリクエストに含まれる経路)
- ・適用されない: 管理画面 API (/auth/*、/projects/* (本エンドポイント自体)、/account-settings/*、/users/*、/billing/* 等)、エンドユーザー側でもメール経由再入室初動の GET /widget/v1/inquiry/reenter?token=... (トークン検証のみで IP 検査をかけると正当な再入室が失敗するため。ただし以降の WebSocket セッション確立時に再検査)

エラー:

- ・400 **VALIDATION_ERROR**(CIDR 形式不正、件数超過、重複行)
- ・403 **E-AUTHZ-OWNER-ONLY**(オーナー以外がアクセス)

1.5.3.5 5.3.4 POST /projects/{id}/widget-key/rotate(ウィジェット鍵ローテーション) | 認証 | 再認証必須 | | 関連画面 | SCR-014 |

リクエスト: { "expiresIn": 7 | 30 | 90 | 180 | 365 } レスポンス (201): { "newKey": "pk_live_xyz789...", "expiresAt": "...", "deprecationWarning": "..."

1.5.4 5.4 FAQ 管理 API

1.5.4.1 5.4.1 GET /faqs?status=draft&projectId=...&keyword=...&cursor=... | 必要権限 | 編集系: オーナー / 該当 PJ の member+(admin または member) / 参照: 同左 |

1.5.4.2 5.4.2 POST /faqs / PATCH /faqs/{id} / DELETE /faqs/{id} 楽観ロック: version 必須。CONFLICT 時 409。

1.5.4.3 5.4.3 POST /faqs/{id}/publish | 関連画面 | SCR-012 |

リクエスト: { "version": 6 } レスポンス (200): { "id": "...", "status": "published", "version": 7, "publishedAt": "..." } エラー: 422 INVALID_STATE (FAQ 自動公開禁止、§ 4.2.1 ガード)、409 CONFLICT

1.5.4.4 5.4.4 POST /faqs/import リクエスト: multipart/form-data { file: <CSV> } レスポンス (202): { "jobId": "job_...", "status": "processing" }

1.5.5 5.5 ウィジェット API

1.5.5.1 5.5.0 共通: プロジェクト単位 IP 許可リスト評価 (FR-179 / FR-330) /widget/v1/* 配下の全エンドポイントは、ドメイン検証およびレート制限と並んでプロジェクト単位 IP 許可リストの評価を実施する。

評価順序(早い段階で拒否することで下流コストを抑制):

1. オリジン (origin ヘッダまたは Referer) が projects.allowed_domains のいずれかと一致 (FR-173)
2. クライアント IP (プロキシ多段の場合は cf-connecting-ip または X-Forwarded-For 最左の信頼可能ホップ) が project_ip_allowlist.cidr のいずれかに含まれる、または当該プロジェクトの IP 許可リストが空 (= 制限なし)
3. レート制限 (NFR-308 / FR-128)
4. 個別エンドポイント認可

評価対象から除外する経路:

- GET /widget/v1/inquiry/reenter?token=... (メール経由再入室の初動。トークン検証のみで IP 検査をかけると正当な再入室が失敗するため。以降の WebSocket セッション確立時に再検査)

拒否時のレスポンス: 403 IP_NOT_ALLOWED (エラー本文には IP を含めない。詳細は 05_エラー設計.md E-BIZ-IP-001 参照)。本拒否はサーバ側で error_logs に件数のみ集計し、エンドユーザーへは「現在ご利用いただけません」のみ表示 (IP 許可リストの存在自体を秘匿)。

1.5.5.2 5.5.1 POST /widget/v1/bootstrap | 認証 | 不要 (ドメイン検証 + IP 許可リスト評価) |

リクエスト:

```
{
  "publicKey": "pk_live_abc123...",
  "origin": "https://example.com"
}
```

レスポンス (200):

```
{
  "sessionToken": "wst_...",
  "expiresIn": 3600,
  "projectConfig": {
    "headerColor": "#06c",
    "placement": "bottom-right",
    "headerTitle": " お問い合わせ",
    "contactEmail": "contact@..."
  }
}
```

エラー: 401 WIDGET_KEY_INVALID / WIDGET_KEY_EXPIRED、403 DOMAIN_NOT_ALLOWED、403 CONTRACT_SUSPENDED

1.5.5.3 5.5.2 POST /widget/v1/ask(質問送信) | 認証 | Bearer session_token | | 機能 ID | F-049-052 |

リクエスト: { "question": " 返品の手続きは?", "projectId": "proj_..." }

レスポンス (200) - answered:

```
{
  "type": "answered",
  "answer": " 返品は商品到着後 7 日以内に...",
  "confidence": 0.78,
  "referencedFaqs": [
```

```

    { "id": "faq_...", "question": " 返品ポリシー", "answer": "..." }
  ],
  "questionLogId": "qlog_..."
}

```

レスポンス (200) - unanswered:

```

{
  "type": "unanswered",
  "reason": "low_confidence" | "no_faq_match" | "pii_detected",
  "questionLogId": "qlog_..."
}

```

エラー: 429 RATE_LIMITED、403 DOMAIN_NOT_ALLOWED

1.5.5.4 5.5.3 POST /widget/v1/inquiries(未解決質問登録) リクエスト:

```

{
  "email": "user@example.com",
  "displayName": " 太郎",
  "questionId": "qlog_...",
  "idempotencyKey": "<UUIDv4>"
}

```

レスポンス (201): { "inquiryId": "inq_...", "inquiryCode": "INQ-20260513-XYZ123", "roomId": "room_..." }

1.5.5.5 5.5.4 POST /widget/v1/chat-rooms/{id}/messages リクエスト: { "body": " メッセージ本文", "idempotencyKey": "<UUIDv4>" } エラー: 429(10/分制限)、400(2,000 字超)

1.5.6 5.6 未解決質問 API

1.5.6.1 5.6.1 GET /inquiries?status=open&projectId=...&cursor=...

1.5.6.2 5.6.2 GET /inquiries/{id} / PATCH /inquiries/{id} PATCH:

```

{ "caseStatus": "resolved", "assigneeAccountId": "..." }

```

1.5.6.3 5.6.3 POST /inquiries/{id}/close(対応不要終了) | 認証 | 再認証必須 | | 重要 | case_status=closed への遷移は admin の明示操作のみ (FR-079)|

リクエスト: { "reason": "resolved_by_faq|customer_resolved|out_of_scope|duplicate" } レスポンス (200): { "id": "...", "caseStatus": "closed", "closedAt": "..." }

1.5.7 5.7 チャット API

1.5.7.1 5.7.1 GET /chat-rooms/{id}/messages / POST /chat-rooms/{id}/messages

1.5.7.2 5.7.2 POST /chat-rooms/{id}/close / POST /chat-rooms/{id}/reopen | 認証 | 再認証必須 | | 条件 (reopen)| 30 日以内の過去クローズのみ |

1.5.8 5.8 利用量・課金 API

1.5.8.1 5.8.1 GET /usage?period=current_month | 機能 ID | F-120 | | 関連画面 | SCR-015 |

レスポンス (200):

```

{
  "period": "2026-05-01T00:00:00Z",
  "questions": { "used": 750, "limit": 1000, "percentage": 75, "resetAt": "2026-06-01T..." },
  "faqs": { "used": 85, "limit": 100, "percentage": 85 },
  "chatRooms": { "used": 28, "limit": 30, "percentage": 93 }
}

```

1.5.8.2 5.8.2 GET /billing/invoices?limit=6 レスポンス (200):

```
{
  "items": [
    {
      "id": "inv_...",
      "date": "2026-05-01T...",
      "amount": 5000,
      "currency": "JPY",
      "status": "paid|failed|draft",
      "pdfUrl": "https://..."
    }
  ]
}
```

pdfUrl は R2 署名 URL(有効期限 5 分)。

1.5.8.3 5.8.3 PATCH /billing/monthly-budget-limit | 認証 | 再認証必須 | | 機能 ID | FR-127 |

リクエスト: { "limitJpy": 50000 } レスポンス (200): { "limitJpy": 50000, "updatedAt": "..." }

1.5.9 5.9 お知らせ受信箱 API

1.5.9.1 5.9.1 GET /me/announcements?cursor=... | 必要権限 | admin 限定 | | 関連画面 | SCR-021 |

レスポンス (200):

```
{
  "items": [
    {
      "id": "ann_...",
      "title": "...",
      "category": "billing|announcement|system",
      "priority": "low|normal|high|critical",
      "bodyHtml": "<p> 本文 </p>",
      "readAt": "..." | null,
      "createdAt": "..."
    }
  ],
  "nextCursor": null
}
```

1.5.9.2 5.9.2 POST /me/announcements/{id}/read レスポンス (200): { "id": "...", "readAt": "..." }

1.5.9.3 5.9.3 GET /me/announcements/unread-summary レスポンス (200): { "unreadsCount": 5, "recent": [...] }

1.5.10 5.10 規約・退会 API

1.5.10.1 5.10.1 GET /terms/current | 認証要否 | 不要 | | 関連画面 | SCR-018 |

1.5.10.2 5.10.2 POST /terms/agree | 関連画面 | SCR-025 | | 関連テーブル | terms_agreements |

1.5.10.3 5.10.3 POST /withdrawal-requests(退会申請) | 認証 | オーナー専有、再認証必須 | | 関連画面 | SCR-024 | | 関連テーブル | withdrawal_requests, accounts |

エラー: 403 PERMISSION_DENIED(メンバー)

1.5.11 5.11 AI 推論 IF(AnswerProvider 抽象化)

```
export type AnswerResult =
  | { kind: 'answered'; answer: string; cited_faq_ids: string[]; confidence: number }
  | { kind: 'unanswerable'; reason_code: 'no_match' | 'low_confidence' | 'contradiction' }
  | { kind: 'error'; reason_code: 'provider_error' | 'timeout' | 'rate_limited' };

export interface AnswerProvider {
  generate(input: {
    question: string;
    candidate_faqs: Array<{ id: string; question: string; answer: string }>;
    policy: { faq_only: true; forbid_new_facts: true; learn: false };
    locale: 'ja-JP';
    timeout_ms: 8000;
  }): Promise<AnswerResult>;

  healthcheck(): Promise<{ ok: boolean; provider: string; model: string }>;
}
```

MVP は Cloudflare Workers AI 実装 (WorkersAIAnswerProvider)。

1.5.12 5.12 メール配信 IF(EmailProvider 抽象化)

```
export interface EmailProvider {
  send(input: {
    to: string; from: string; reply_to?: string; subject: string;
    html: string; text?: string; headers?: Record<string, string>;
    idempotency_key: string;
  }): Promise<{ message_id: string }>;

  verifyWebhook(headers: Headers, body: string): Promise<{
    valid: boolean;
    event_type: 'sent' | 'delivered' | 'bounced' | 'complained' | 'failed' | 'opened' | 'clicked';
    message_id: string;
    timestamp: string;
  }>;
}
```

MVP は Resend 実装 (ResendEmailProvider)。

1.5.13 5.13 連携 IF(受信側、メイン側エンドポイント)

1.5.13.1 5.13.1 IF #1 契約停止イベント受信 (顧客 → メ)

POST /internal/admin-integration/v1/owner/suspend

認証: mTLS + JWT(aud=main、TTL 5min)

リクエスト:

```
{
  "operationId": "01HZ...",
  "ownerAccountId": "01HZ...",
  "targetStatus": "suspended",
  "reason": "payment_failed|terms_violation|manual_admin",
  "occurredAt": "2026-05-13T10:00:00Z",
  "operatorAccountId": "..."
}
```

ヘッダ: Idempotency-Key: (operation_id, owner_account_id, target_status)

レスポンス (200): { "ok": true, "newStatus": "suspended", "appliedAt": "..." }

動作:

1. JWT + Idempotency-Key 検証
2. accounts.contract_status(オーナー行) 更新

3. KV キャッシュ無効化(セッション/KV/レート制限)
4. reason に応じてセッション処理(operator_manual / tos_violation は 5 秒以内 に全セッション無効化)

エラー: 401 JWT_INVALID、409 CONFLICT

1.5.13.2 5.13.2 IF #2 強制ログアウト受信

POST /internal/admin-integration/v1/owner/forced-logout

リクエスト:

```
{
  "operationId": "...",
  "ownerAccountId": "...",
  "scope": "all|single_account",
  "accountId": "...",
  "reason": "..."
}
```

動作: sessions.revoked_at 設定 + KV キャッシュ強制 invalidate(5 秒以内)

1.5.13.3 5.13.3 IF #4 復元実行受信

POST /internal/admin-integration/v1/owner/restore

認証: mTLS + JWT + 再認証トークン

1.5.13.4 5.13.4 IF #5 レート制限上書き受信

POST /internal/admin-integration/v1/rate-limit/override

リクエスト:

```
{
  "ownerAccountId": "...",
  "overrideId": "...",
  "limits": [{ "kind": "ask", "limit": 30, "windowSec": 60, "validUntil": "..." }],
  "reason": "...",
  "operatorAccountId": "..."
}
```

動作:

1. JWT + Idempotency-Key 検証
2. owner_quota_overrides テーブル更新
3. KV キー ratelimit:{ownerAccountId}:{kind} 更新(即時反映)

1.5.13.5 5.13.5 IF #6 AI しきい値上書き受信

POST /internal/admin-integration/v1/threshold/update

リクエスト:

```
{
  "ownerAccountId": "...",
  "version": "2026-05-13T10:00:00Z",
  "projectId": null,
  "confidenceThreshold": 0.65,
  "relevanceThreshold": 0.55,
  "scope": "owner|project",
  "operatorAccountId": "..."
}
```

動作:

1. JWT + Idempotency-Key 検証
2. KV ai_threshold:{ownerAccountId}:{projectId} 更新(60s TTL)
3. ai_threshold_persistent_cache に永続化

4. IF #12 経由で管理者ユーザーに inbox 配信

1.5.13.6 5.13.6 IF #7 お知らせ生成受信

POST /internal/admin-integration/v1/announcement/inbound

リクエスト:

```
{
  "announcementId": "...",
  "ownerAccountId": null,
  "title": "...",
  "bodyHtml": "<p>...</p>",
  "importance": "low|normal|high|critical",
  "audienceOwnerAccountIds": null,
  "publishedAt": "...",
  "operatorAccountId": "..."
}
```

動作:

1. JWT + Idempotency-Key 検証
2. **service_announcements** レコード作成
3. **announcement_recipients** を宛先範囲に応じて生成
4. announcement-fanout-queue に投入、**inbox_messages** へ fan-out

1.5.13.7 5.13.7 IF #8 監視メトリクス取得 (送信側、メ → 顧管)

GET https://<admin-host>/internal/main-integration/v1/metrics?window=5m|1h|24h

認証: mTLS + JWT(aud=admin)

レスポンス (200):

```
{
  "window": "5m",
  "since": "...",
  "until": "...",
  "owners": [
    {
      "ownerAccountId": "...",
      "errorRate5xx": 0.002,
      "aiInferenceP95Ms": 1234,
      "queueDlqCount": 0,
      "emailBounceRate": 0.01,
      "emailComplaintRate": 0.0005,
      "billingWebhookFailures": 0,
      "aiAnswerableRate": 0.85
    }
  ],
  "global": { "d1UsageRatio": 0.65 }
}
```

1.5.13.8 5.13.8 IF #9 不正利用検知通知 (送信側、メ → 顧管)

POST https://<admin-host>/internal/main-integration/v1/fraud/detected

リクエスト: { "detectionId": "...", "ownerAccountId": "...", "pattern": "...", "details": {...} }

DLQ 上限: 200/30 分

1.5.13.9 5.13.9 IF #10 課金 Webhook 受信 (顧管 → メ)

POST /internal/admin-integration/v1/billing-webhook/forward

リクエスト:

```
{
  "stripeEventId": "evt_...",
  "eventType": "invoice.paid|invoice.payment_failed|customer.subscription.created|...",
  "ownerAccountId": "...",
  "payload": { /* Stripe イベント */ },
  "receivedAt": "...",
  "signatureVerifiedAt": "..."
}
```

動作:

1. JWT 検証 + event_id 冪等性
2. Stripe イベント種別に応じてハンドラ分岐:
 - **invoice.paid**: **billing_invoices.status**='paid'、suspended → active 復帰
 - **invoice.payment_failed**: 通知、7 日猶予タイマー開始
 - **customer.subscription.created**: **billing_subscriptions** 作成
 - **charge.refunded**: 訂正請求行記録
3. IF #1 経由で **accounts.contract_status** 反映

Idempotency-Key: event_id(Stripe evt_*) DLQ 滞留上限: 1000/24h

1.5.13.10 5.13.10 IF #12 運営者操作通知受信

POST /internal/admin-integration/v1/admin-operation/notify

リクエスト:

```
{
  "operationId": "...",
  "ownerAccountId": "...",
  "operationKind": "owner.restore_data|owner.suspend|rate_limit.override|...",
  "occurredAt": "...",
  "actorOperatorId": "...",
  "ticketId": "...",
  "summary": " 削除データを復元しました"
}
```

動作:

1. JWT + Idempotency-Key 検証
2. **inbox_messages** に重要度 **high** で生成 (管理者 inbox 表示)
3. メール通知 Queue 投入 (**critical** の場合はメール強制送信)

1.5.14 5.14 外部 Webhook(Resend 等)

1.5.14.1 5.14.1 POST /webhooks/resend 認証: Resend 署名検証 (HMAC-SHA256)

リクエスト (Resend 仕様準拠):

```
{
  "type": "email.delivered|email.bounced|email.complained|email.delivery_delayed",
  "data": {
    "message_id": "...",
    "to": "...",
    "subject": "...",
    "timestamp": "..."
  }
}
```

動作:

1. 署名検証 (失敗時は 401)
2. **notification_logs.delivery_state** を遷移
3. **bounced / complained** の場合は **email_suppression_list** に追加 (全契約横断)

エラー: 401 **SIGNATURE_INVALID**、200 + 既存処理結果 (冪等性違反)

1.6 6. 連携 IF 通信仕様一覧

IF#	連携先	方向	認証	Idempotency	DLQ 上限	TO	主管
1	顧管	→ ×	mTLS + JWT(aud=main)	(operation_id, owner_account_id, target_status)	100/30分	5s	受信
2	顧管	→ ×	mTLS + JWT	(operation_id, owner_account_id)	100/30分	3s	受信
4	顧管	→ ×	mTLS + JWT + 再認証	(restore_op_id)	50/24h	10s	受信
5	顧管	→ ×	mTLS + JWT	(owner_id, override_id)	50/30分	3s	受信
6	顧管	→ ×	mTLS + JWT	(owner_id, scope, version)	50/30分	3s	受信
7	顧管	→ ×	mTLS + JWT	(announcement_id, owner_id)	1000/h	30s	受信
8	顧管	× →	mTLS + JWT(aud=admin)	(metric_window_id)	n/a	10s	送信
9	顧管	× →	mTLS + JWT(aud=admin)	(detection_id)	200/30分	5s	送信
10	Stripe → 顧管 → ×	→ ×	Stripe 署名 + mTLS	event_id	1000/24h	10s	受信
12	顧管	→ ×	mTLS + JWT	(op_id, owner_id)	200/30分	5s	受信

詳細は運営者側 [03_API 設計書.md §5.8](#) を参照。

1.7 7. 未決事項

No	内容	確認先	期限	ステータス
1	IF #3 / #11 の用途	要件側で予約済み、MVP 範囲外	Future	既決

1.8 8. 変更履歴

日付	版数	変更内容	変更者
2026-05-17	1.0	初版作成	claude